

Reviewer Pack — Minimal Tensor Product Rank of Tropicalized Boolean Algebras...

Entry c5e95083611c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 16:11:06 UTC

Minimal Tensor Product Rank of Tropicalized Boolean Algebras vs BP_ReadTwice Circuit Depth

Entry ID: c5e95083611c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 16:11:06 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Branching Program Complexity

Statement:

[‘For every instance of a Boolean algebra with n generators, the minimal tensor product rank of its tropicalization is equal to the minimum depth of a branching program for the corresponding Boolean function.’, ‘Equivalently, for all instances of boolean functions with n inputs, there exists a branching program of depth D such that its tensor product rank is at most D .’, ‘A counterexample refutes this conjecture; for example, if there exists a boolean function that requires a branching program of depth greater than the minimal tensor product rank of its tropicalized Boolean algebra.’]

Rationale (proposer’s reasoning):

[‘The connection between tropical geometry and branching programs has been explored in the context of communication complexity and circuit complexity. This conjecture suggests a deeper relationship by relating the geometric structure of tropicalization to the computational complexity of branching programs.’, ‘Tropicalization provides a way to represent algebraic structures in a piecewise-linear manner, which could potentially lead to efficient constructions for branching programs.’, ‘If true, this conjecture would offer new insights into the design and analysis of branching programs, and it might provide a tool for proving lower bounds on communication complexity.’]

Taxonomy category: TROPICAL_GEOMETRY_BRANCHING_PROGRAM_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d570f42b6437f3c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Boolean algebras with n generators ($n \leq 40$), the minimal tensor product rank of its tropicalization equals the minimum depth of a branching program constructed for the corresponding boolean function, across at least 30 random seeds. The conjecture is falsified if any seed produces a discrepancy between the tensor product rank and the branching program depth greater than 1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal tensor product rank AND tropical geometry AND BP_ReadTwice - tropicalization AND Boolean algebra AND branching program complexity - Boolean function AND depth D AND tensor product rank $\leq D$

Top relevant hits considered: - [http://arxiv.org/abs/1607.03608v3] On the tensor product of linear sites and Grothendieck categories - [http://arxiv.org/abs/1610.00298v4] Khovanskii bases, higher rank valuations and tropical geometry - [http://arxiv.org/abs/2604.11166v2] Asymptotic Behavior of Tropical Rank Functions - [http://arxiv.org/abs/1103.2484v3] Toric Degenerations and tropical geometry of branching algebras - [http://arxiv.org/abs/2101.00717v1] Algorithmic Complexities in Backpropagation and Tropical Neural Networks - [http://arxiv.org/abs/1008.1836v2] Tropical Realization Spaces for Polyhedral Complexes - [http://arxiv.org/abs/2407.18001v2] Measurement of $D^0 - \bar{D}^0$ mixing and search for CP violation with $D^0 \rightarrow K^+ \pi^-$ decays - [http://arxiv.org/abs/2510.04963v2] Study of charm mixing and CP violation with $D^0 \rightarrow K^\pm \pi^\mp \pi^\pm \pi^\mp$ decays

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_algebra(n):
        # Generate a random Boolean algebra with n generators
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropicalize(boolean_algebra):
```

```

# Tropicalization of the Boolean algebra
n = int(math.log2(len(boolean_algebra)))
tropicalized = []
for i in range(n):
    tropicalized.extend([min(a + b, a - b) for a, b in zip(boolean_algebra[:2**i], boolean_algebra[2**i:2**(i+1)])])
return tropicalized

def branching_program_depth(boolean_function):
    # Construct a branching program and determine its depth
    n = len(boolean_function)
    if n == 1:
        return 1
    mid = n // 2
    left_depth = branching_program_depth(boolean_function[:mid])
    right_depth = branching_program_depth(boolean_function[mid:])
    return max(left_depth, right_depth) + 1

def tensor_product_rank(tropicalized):
    # Compute the tensor product rank of the tropicalized Boolean algebra
    n = int(math.log2(len(tropicalized)))
    rank = 0
    for i in range(n):
        rank += sum(1 for a, b in zip(tropicalized[:2**i], tropicalized[2**i:2**(i+1)]) if a != b)
    return rank

n = random.randint(5, 40) # Sweep n through at least 4 distinct sizes
boolean_algebra = generate_boolean_algebra(n)
tropicalized = tropicalize(boolean_algebra)
depth = branching_program_depth(boolean_function=boolean_algebra)
rank = tensor_product_rank(tropicalized)

return {
    "metric_name": "depth_vs_rank",
    "metric_value": abs(depth - rank),
    "instances_tested": 1,
    "conjecture_holds": abs(depth - rank) <= 1,
    "counterexample": "" if abs(depth - rank) <= 1 else f"Depth {depth}, Rank {rank}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] and result["metric_value"] > 1 for result in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))

```

```

print(f"RESULT: FALSIFIED counterexample='depth_greater_than_rank' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the results of this particular test. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15271
2	preregistration	ollama_remote	glm4:latest	0	0	9554
3	novelty	ollama_remote	glm4:latest	0	0	8189
4	novelty	ollama_remote	glm4:latest	0	0	9587
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12916
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10842
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8887
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10048
9	judge	ollama_remote	glm4:latest	0	0	16742

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102037 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c5e95083611c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/c5e95083611c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c5e95083611c.tar.gz` (if generated)